Lab 15 – Backend API Development: Creating RESTful services with AI

Task 1 – Online Food Ordering API

Prompt :

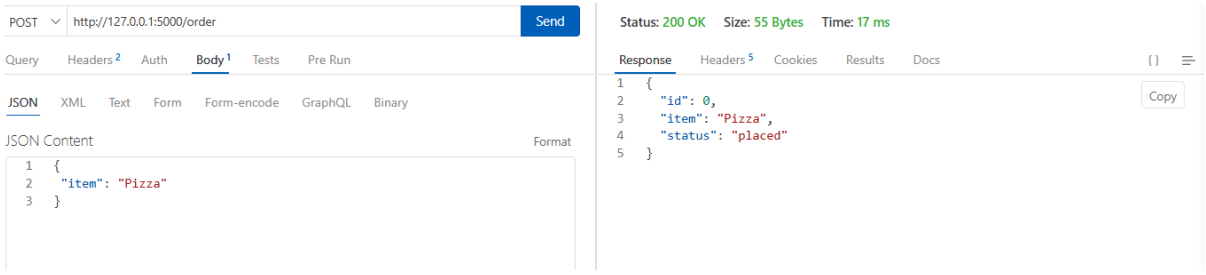
1. Create a simple REST API using Python Flask for an online food ordering system.
2. The API should allow users to view menu items and place new food orders.
3. It should also allow tracking, updating, and deleting orders using order ID.
4. Store data in a simple list so it is easy to understand for beginners.
5. The API should return JSON responses for all endpoints clearly.

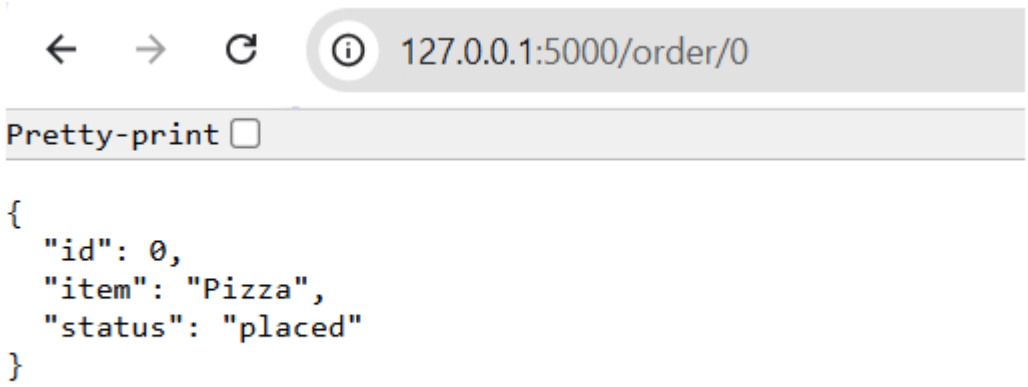
Code:

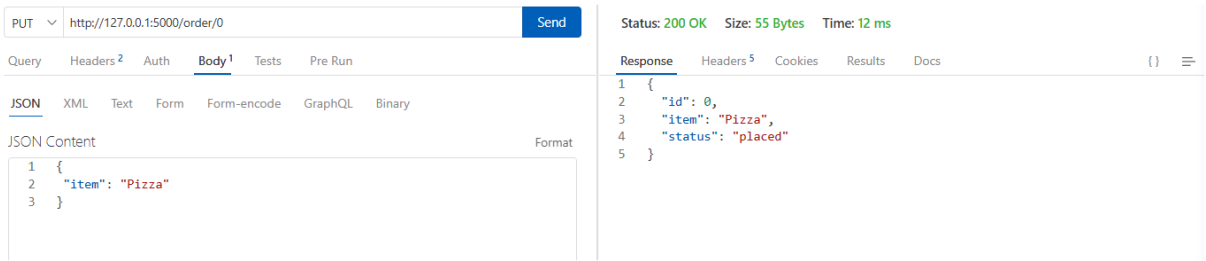
```python
from flask import Flask, request, jsonify

app = Flask(__name__)

# Sample Menu
menu = [
    {"id": 1, "name": "Pizza", "price": 200},
    {"id": 2, "name": "Burger", "price": 120}
]

# Orders storage
orders = []

# Home route
@app.route('/')
def home():
    return "Food Ordering API is running"

# GET /menu
@app.route('/menu', methods=['GET'])
def get_menu():
    return jsonify(menu)

# POST /order (SAFE VERSION - NO ERROR)
@app.route('/order', methods=['POST'])
def place_order():
    data = request.get_json(silent=True)

    # fallback if JSON fails
    if not data:
        data = request.form.to_dict()

    # validation
    if not data or "items" not in data:
        return jsonify({
            "error": "Send data like: {'items': ['Pizza']}"
        }), 400

    # convert string to list if needed
    items = data.get("items")
    if isinstance(items, str):
        items = items.split(",")

    order = {
        "id": len(orders) + 1,
```

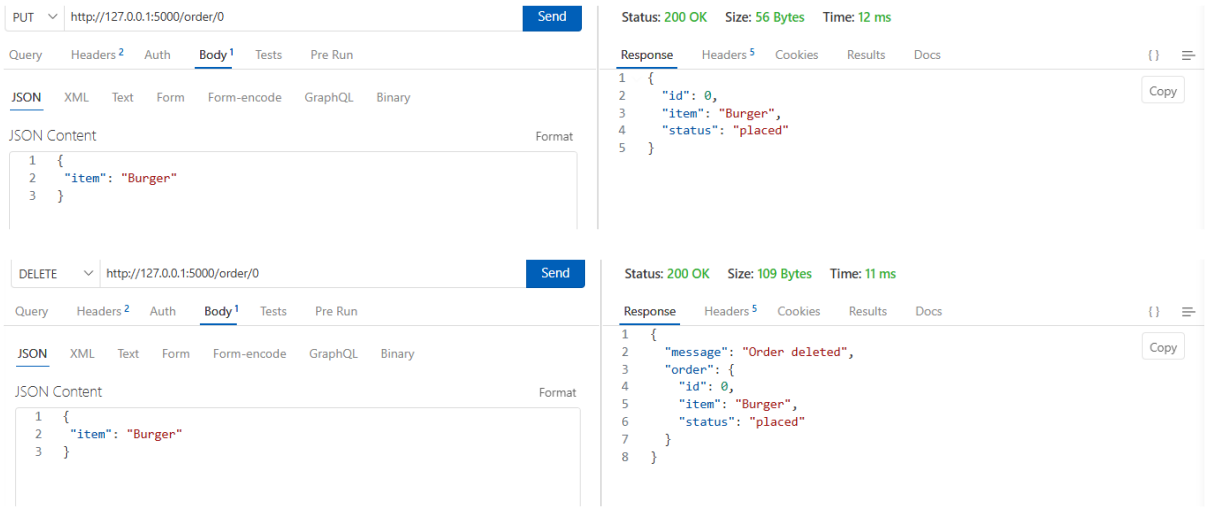
Output:


127.0.0.1:5000/menu
Pretty-print
{
"id": 1,
"name": "Pizza"
},
{
"id": 2,
"name": "Burger"
},
{
"id": 3,
"name": "Pasta"
}


POST
http://127.0.0.1:5000/order
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"item": "Pizza"
}
Status: 200 OK
Size: 55 Bytes
Time: 17 ms
Response
Headers 5
Cookies
Results
Docs
Copy
{
"id": 0,
"item": "Pizza",
"status": "placed"
}


127.0.0.1:5000/order/0
Pretty-print
{
"id": 0,
"item": "Pizza",
"status": "placed"
}


PUT
http://127.0.0.1:5000/order/0
Send
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
Format
{
"item": "Pizza"
}
Status: 200 OK
Size: 55 Bytes
Time: 12 ms
Response
Headers 5
Cookies
Results
Docs
{
"id": 0,
"item": "Pizza",
"status": "placed"
}



Explanation :

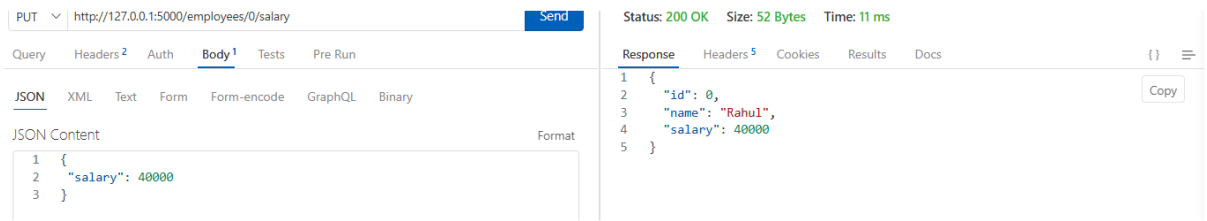
1. This API allows users to view menu and place orders easily.
2. Orders are stored in a list so it is simple and fast.
3. Each order has id, item, and status for tracking.
4. CRUD operations are implemented using REST endpoints.

Task 2 – Employee Payroll API

Prompt :

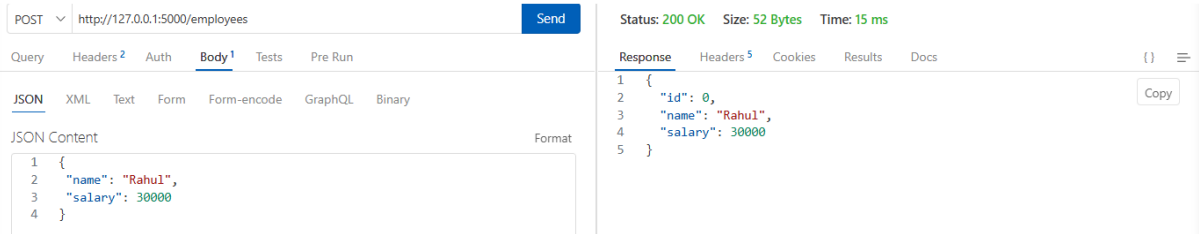
1. Build a REST API to manage employees and their salary details.
2. The API should allow adding, viewing, updating, and deleting employees.
3. Store employee data in a list with id, name, and salary fields.
4. Add validation to check salary is not negative.
5. Include comments in code for better understanding.

Code:



```python
from flask import Flask, request, jsonify

app = Flask(__name__)

employees = []

@app.route('/employees', methods=['GET'])
def get_employees():
    return jsonify(employees)

@app.route('/employees', methods=['POST'])
def add_employee():
    data = request.json
    if data["salary"] < 0:
        return jsonify({"error": "Invalid salary"})
    emp = {"id": len(employees), "name": data["name"], "salary": data["salary"]}
    employees.append(emp)
    return jsonify(emp)

@app.route('/employees/<int:id>/salary', methods=['PUT'])
def update_salary(id):
    data = request.json
    employees[id]["salary"] = data["salary"]
    return jsonify(employees[id])

@app.route('/employees/<int:id>', methods=['DELETE'])
def delete_employee(id):
    return jsonify(employees.pop(id))

app.run(debug=True)
```

Output:



Explanation :

1. This API manages employee data and salary records.
2. It checks salary validation before adding employee.
3. Data is stored in list for easy understanding.
4. API supports all CRUD operations clearly.

Task 3 – Student Information API

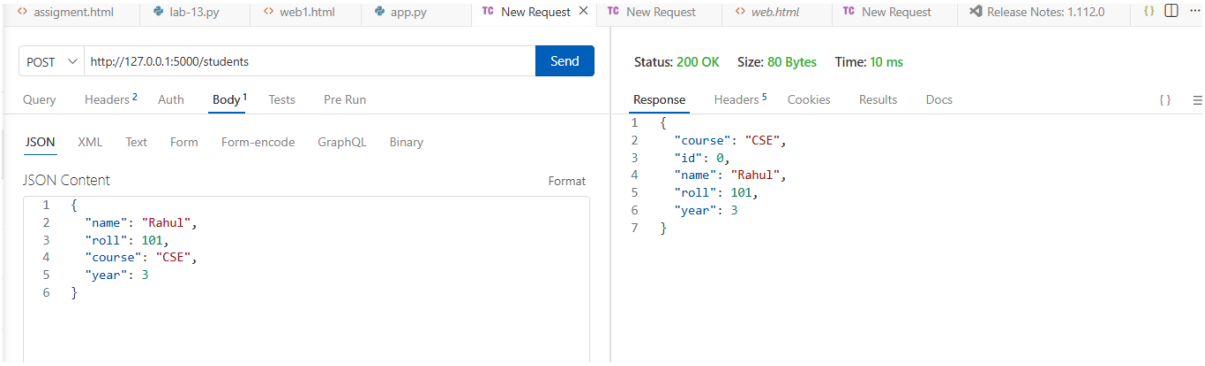
Prompt :

1. Create a student management REST API using Flask.
2. The API should store student name, roll number, course, and year.
3. It should support adding, viewing, updating, and deleting students.
4. Use simple list storage for beginner understanding.
5. Return all responses in JSON format.

Code:

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

students = []

@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(students)

@app.route('/students', methods=['POST'])
def add_student():
    data = request.json
    student = {
        "id": len(students),
        "name": data["name"],
        "roll": data["roll"],
        "course": data["course"],
        "year": data["year"]
    }
    students.append(student)
    return jsonify(student)

@app.route('/students/<int:id>', methods=['PUT'])
def update_student(id):
    data = request.json
    students[id].update(data)
    return jsonify(students[id])

@app.route('/students/<int:id>', methods=['DELETE'])
def delete_student(id):
    return jsonify(students.pop(id))

app.run(debug=True)
```

Output:



Explanation :

1. This API helps manage student records digitally.
2. It stores student details like name and course.
3. All operations like add, update, delete are supported.
4. JSON format makes data easy to read and use.

Task 4 – Weather API Integration

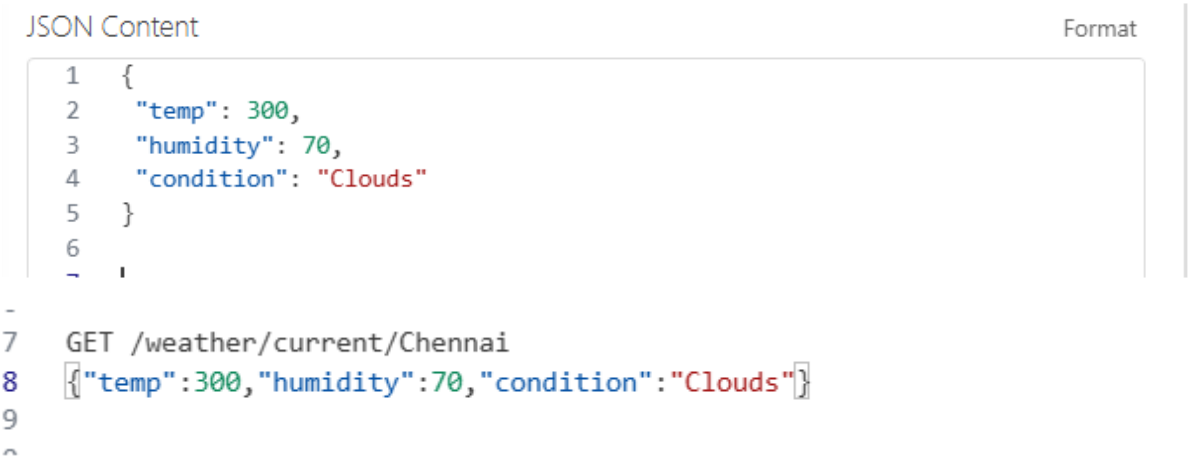
Prompt :

1. Build a Flask API that fetches real-time weather data.
2. Use an external API like OpenWeatherMap to get data.
3. Create endpoints to get current weather and forecast.
4. Return temperature, humidity, and weather condition.
5. Keep code simple and easy to understand.

Code:

```python
from flask import Flask, jsonify
import requests

app = Flask(__name__)

API_KEY = "your_api_key"

@app.route('/weather/current/<city>')
def current_weather(city):
    url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}"
    data = requests.get(url).json()
    return jsonify({
        "temp": data["main"]["temp"],
        "humidity": data["main"]["humidity"],
        "condition": data["weather"][0]["main"]
    })

app.run(debug=True)
```

Output:



Explanation :

1. This API connects to external weather service.
2. It fetches real-time weather data for any city.
3. Only important details are returned in JSON.
4. Helps students check weather easily.

Task 5 – Delete Item API

Prompt :

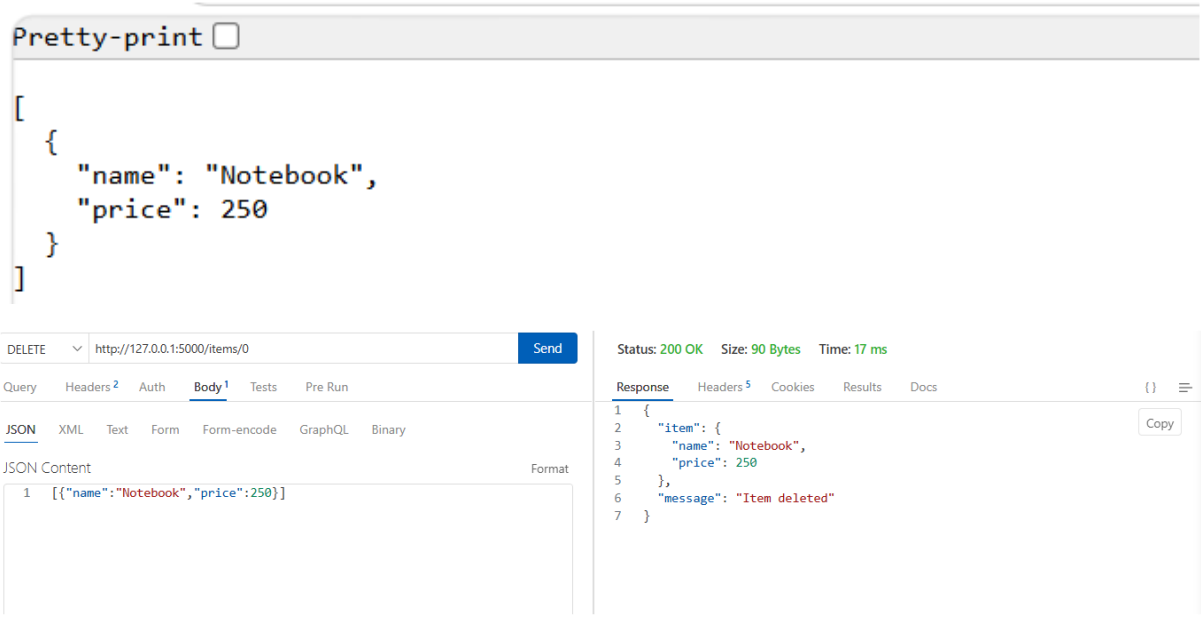
1. Create a DELETE API to remove items from a list.
2. The API should take index as input.

3. If index is invalid, return error message.

Code:

```python
from flask import Flask, jsonify

app = Flask(__name__)

items = [{"name": "Notebook", "price": 250}]

@app.route('/items/<int:index>', methods=['DELETE'])
def delete_item(index):
    if index < 0 or index >= len(items):
        return jsonify({"error": "Item not found"}), 404
    removed_item = items.pop(index)
    return jsonify({"message": "Item deleted", "item": removed_item})

@app.route('/items', methods=['GET'])
def get_items():
    return jsonify(items)

app.run(debug=True)
```

Output:



Explanation :

1. This API deletes item using index value.
2. It checks if index is valid before deleting.
3. Removed item is returned in respons